

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №11
з курсу «Хмарні технології та великі дані»
на тему:
«Написати Docker Compose файл для двох сервісів»

Виконав:
студент групи ФеІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	4
Критерії виконання роботи.....	4
Теоретичні відомості	4
Структура програмної реалізації	5
Тестування програмної реалізації.....	5
Висновок	9
Код програми	10

Тема

Декларативна оркестрація мульти-контейнерних застосунків: проектування архітектури взаємодії веб-сервісу та системи збереження даних за допомогою специфікації Docker Compose.

Мета

Дослідження принципів декларативного опису та автоматизації розгортання мульти-сервісних розподілених середовищ засобами Docker Compose. Опанування механізмів формування маніфесту `compose.yaml`, конфігурації мікросервісної взаємодії через ізольовану програмно-визначену мережу (bridge network), встановлення суворої черговості запуску контейнерів через директиву залежностей `depends_on`, маршрутизації портів між хостовою системою та внутрішніми сервісами, а також комплексне тестування життєвого циклу кластера (валідація, розгортання, збір журналів та коректне виведення з експлуатації).

Вхідні дані

Параметр / Директива	Значення / Тип	Призначення
Специфікація конфігурації	<code>compose.yaml</code>	Декларативний маніфест структури сервісів, мереж та томів мульти-контейнерного додатку.
Сервіс <code>app</code>	<code>python:3.11-alpine (Flask)</code>	Користувачський веб-сервіс, що обробляє вхідні HTTP-запити та модифікує стан лічильника в базі даних.
Сервіс <code>redis</code>	<code>redis:7-alpine</code>	Зовнішнє швидкісне in-memory сховище типу «ключ-значення», що забезпечує збереження стану лічильника.
Портова трансляція	<code>8080:8080 (-p)</code>	Прокидання порту 8080 хостової машини на робочий порт 8080 веб-сервера у внутрішньому просторі контейнера.
Механізм зв'язування	<code>depends_on</code>	Декларативне правило черговості запуску, що забезпечує ініціалізацію сховища Redis до старту веб-застосунку.
Віртуальна мережа	<code>app-network (driver: bridge)</code>	Ізольований програмний мережевий простір для автоматичного DNS-резолвінгу імен сервісів між контейнерами.

Завдання

- спроектувати декларативний маніфест `compose.yaml` у кореновому каталозі проєкту із визначенням двох взаємозалежних сервісів: веб-застосунку `app` та `in-memory` сховища `redis`.
- реалізувати механізм прямого мапінгу вхідного порту `8080:8080` для забезпечення доступу до сервісу `app` із хостової системи.
- забезпечити гарантований порядок ініціалізації середовища шляхом конфігурації залежності `depends_on` між застосунком та сховищем.
- об'єднати обидва сервіси в спільну внутрішню `bridge`-мережу для підтримки автоматичного `Service Discovery` за символьним хостнеймом `redis`.
- здійснити валідацію структури маніфесту через утиліту `docker-compose config` та провести запуск стека в `detached`-режимі.
- провести верифікацію доступності веб-вузла, перевірити коректність збільшення лічильника в `Redis` через серію HTTP-запитів `curl` та проаналізувати консольні логи сервісів.
- продемонструвати процедуру коректного завершення роботи та вивільнення мережевих ресурсів командою `docker-compose down`.

Критерії виконання роботи

- *Декларативність та валідність*: конфігурація проєкту повністю описана у файлі `compose.yaml` та успішно проходить синтаксичну верифікацію без попереджень і помилок парсингу.
- *Автономність взаємодії сервісів*: контейнер веб-застосунку здатний детерміновано виявляти вузол `Redis` за його службовим іменем завдяки вбудованому механізму `DNS Docker Compose` без використання статичних IP-адрес.
- *Контроль залежностей*: застосунок не стартує раніше початку ініціалізації сховища даних, запобігаючи виникненню помилок розриву початкового з'єднання (`Connection Refused`).
- *Ізоляція та безпека портів*: внутрішній порт бази даних `6379` залишається закритим для зовнішньої мережі хоста і доступний виключно членам віртуальної мережі `app-network`, тоді як назовні виставляється лише прикладний порт `8080`.

Теоретичні відомості

Зі збільшенням складності програмних рішень сучасна архітектура систем переходить від монолітних модулів до розподілених мікросервісів, де кожен функціональний компонент (бізнес-логіка, бази даних, кеш, брокери повідомлень) ізолюється у власному контейнері. Ручне управління десятками контейнерів через стандартні низькорівневі команди запуску вимагає постійного контролю параметрів мережі та черговості, що створює ризики людського фактора і суттєво ускладнює тиражування середовища.

Інструмент Docker Compose вирішує проблему оркестрації за допомогою парадигми «Інфраструктура як код» (Infrastructure as Code, IaC). Він дає змогу в єдиному маніфесті формату YAML описати зв'язки між сервісами, правила маршрутизації мережевого трафіку, монтування файлових томів (volumes) та змінні оточення.

Однією з фундаментальних переваг Docker Compose є автоматичне розгортання окремої віртуальної мережі типу Bridge для кожного проєкту. Усередині цієї ізольованої мережі активується вбудований DNS-сервер Docker, завдяки якому кожному сервісу автоматично призначається мережевий псевдонім (hostname), еквівалентний назві сервісу в конфігурації. Це дозволяє коду застосунку звертатися до сховища даних за стабільним символьним іменем (redis:6379), уникаючи динамічних IP-адрес, що генеруються під час перестворення контейнерів.

Структура програмної реалізації

Програмний стенд реалізовано за модульним принципом відповідно до вимог лабораторної роботи:

- *compose.yaml*: декларативний маніфест специфікації Docker Compose, який визначає властивості контейнерів *app* та *redis*, директиви залежностей *depends_on*, конфігурацію порту 8080:8080 та спільну bridge-мережу *app-network*.
- *app.py*: скрипт веб-застосунку на мові Python із використанням бібліотеки Flask та драйвера *redis*. Реалізує REST-ендпоінт */*, що виконує атомарний інкремент ключа *counter* у пам'яті Redis при кожному підключенні та повертає згенеровану HTML-відповідь.

Тестування програмної реалізації

Етап 1. Перевірка валідності конфігураційного маніфесту

Мета: Верифікація синтаксичної коректності структури файлу *compose.yaml* та інтерпретації параметрів парсером Docker Compose.

У середовищі термінала виконано команду перевірки структури файлу:

docker-compose config

```

/mnt/d/4_1/Cloud_Technologies/Lab_11 # docker-compose config
name: lab_11
services:
  app:
    command:
      - sh
      - -c
      - pip install --no-cache-dir flask redis && python app.py
    container_name: lab11-web-app
    depends_on:
      redis:
        condition: service_started
        required: true
    image: python:3.11-alpine
    networks:
      app-network: null
    ports:
      - mode: ingress
        target: 8080
        published: "8080"
        protocol: tcp
    restart: always
    volumes:
      - type: bind
        source: /mnt/d/4_1/Cloud_Technologies/Lab_11
        target: /app
        bind: {}
    working_dir: /app
  redis:
    container_name: lab11-redis
    image: redis:7-alpine
    networks:
      app-network: null
    restart: always
networks:
  app-network:
    name: lab_11_app-network
    driver: bridge

```

Рис. 1 — Перевірка валідності конфігураційного маніфесту

Аналіз результату: Утиліта успішно здійснила синтаксичний розбір маніфесту та повернула нормалізоване дерево проєкту. Усі директиви зв'язування томів, портової переадресації та оголошення віртуальної мережі валідовані без синтаксичних помилок і конфліктів.

Етап 2. Розгортання та запуск мульти-контейнерного стека

Мета: Декларативна ініціалізація інфраструктури, створення мережевого мосту та запуск сервісів у фоновому режимі із дотриманням черговості.

Командою фонового запуску розгорнуто стек сервісів, після чого перевірено їхні системні статуси:

docker-compose up -d

docker-compose ps

Після аналізу логів виконано процедуру повної зупинки та видалення середовища:

docker-compose down

```
lab11-web-app | Collecting werkzeug>=3.1.0 (from flask)
lab11-web-app |   Downloading werkzeug-3.1.9-py3-none-any.whl.metadata (4.1 kB)
lab11-web-app | Downloading flask-3.1.3-py3-none-any.whl (103 kB)
lab11-web-app |   _____ 103.4/103.4 kB 2.3 MB/s eta 0:00:00
lab11-web-app | Downloading redis-8.1.0-py3-none-any.whl (560 kB)
lab11-web-app |   _____ 560.6/560.6 kB 7.1 MB/s eta 0:00:00
lab11-web-app | Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
lab11-web-app | Downloading click-8.5.0-py3-none-any.whl (125 kB)
lab11-web-app |   _____ 125.3/125.3 kB 13.0 MB/s eta 0:00:00
lab11-web-app | Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
lab11-web-app | Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
lab11-web-app |   _____ 134.9/134.9 kB 9.7 MB/s eta 0:00:00
lab11-web-app | Downloading markupsafe-3.0.4-cp311-cp311-musllinux_1_2_x86_64.whl (22 kB)
lab11-web-app | Downloading werkzeug-3.1.9-py3-none-any.whl (228 kB)
lab11-web-app |   _____ 228.7/228.7 kB 10.9 MB/s eta 0:00:00
lab11-web-app | Installing collected packages: redis, markupsafe, itsdangerous, click, blinker, wer
kzeug, jinja2, flask
lab11-web-app | Successfully installed blinker-1.9.0 click-8.5.0 flask-3.1.3 itsdangerous-2.2.0 jin
ja2-3.1.6 markupsafe-3.0.4 redis-8.1.0 werkzeug-3.1.9
lab11-web-app | WARNING: Running pip as the 'root' user can result in broken permissions and confli
cting behaviour with the system package manager. It is recommended to use a virtual environment inst
ead: https://pip.pypa.io/warnings/venv
lab11-web-app |
lab11-web-app | [notice] A new release of pip is available: 24.0 -> 26.2.1
lab11-web-app | [notice] To update, run: pip install --upgrade pip
lab11-web-app | * Serving Flask app 'app'
lab11-web-app | * Debug mode: off
lab11-web-app | WARNING: This is a development server. Do not use it in a production deployment. Us
e a production WSGI server instead.
lab11-web-app | * Running on all addresses (0.0.0.0)
lab11-web-app | * Running on http://127.0.0.1:8080
lab11-web-app | * Running on http://172.18.0.3:8080
lab11-web-app | Press CTRL+C to quit
lab11-web-app | 172.18.0.1 - - [03/Oct/2026 15:52:42] "GET / HTTP/1.1" 200 -
lab11-web-app | 172.18.0.1 - - [03/Oct/2026 15:52:48] "GET / HTTP/1.1" 200 -
lab11-web-app | 172.18.0.1 - - [03/Oct/2026 15:52:50] "GET / HTTP/1.1" 200 -
```

Рис. 5 — Моніторинг системних логів

Аналіз результату: У зведеному журналі зафіксовано штатні записи ініціалізації Redis та протоколювання оброблених GET-запитів веб-сервером без помилок доступу чи мережевих збоїв. Команда *docker-compose down* безпечно завершила виконання контейнерних процесів через надсилання сигналу SIGTERM та видалила внутрішню віртуальну мережу, звільнивши всі виділені системні ресурси.

Висновок

У ході виконання лабораторної роботи було досліджено теоретичні засади та практично реалізовано механізми декларативної оркестрації мульти-контейнерних застосунків за допомогою інструменту Docker Compose.

Було спроектовано конфігураційний маніфест `compose.yaml`, який об'єднав користувацький веб-застосунок на Python та базу даних Redis у межах спільної ізольованої bridge-мережі. На практиці підтверджено ефективність концепції Service Discovery: завдяки автоматичній роботі внутрішнього DNS-сервера взаємодія між компонентами відбувалася за символьним ім'ям сервісу без прив'язки до статичних IP-адрес. Використання директиви `depends_on` дозволило запобігти збоєм підключення під час холодного старту шляхом встановлення суворої послідовності ініціалізації.

Проведене тестування за допомогою curl-запитів підтвердило коректність прокидання зовнішнього порту 8080 та збереження стану даних у сховищі Redis. Аудит журналів та успішне очищення ресурсів командою `docker-compose down` підтвердили стабільність, ізольованість і відповідність створеного проєкту всім поставленим критеріям.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_11 та має повну інструкцію запуску даної програми.